

Toka Priyanka

tokapriyanka@gmail.com

AI/ML intern

Role: Computer Vision

AGNOSTIC-v3.2

Introduction

Agnostic representation is one of the preprocessing stages used in virtual try-on systems. The main objective of Agnostic-v3.2 is to remove clothing appearance information while preserving the body structure of the user.

In virtual try-on systems, clothing information from the original image may interfere with new garment placement. Therefore, an agnostic image is generated where the upper garment region is hidden while body posture remains unchanged.

In this implementation, webcam input is captured and MediaPipe pose estimation is used to identify body landmarks. The upper body area including shoulders, arms and torso is masked using gray color.

Algorithm

- Step 1 : Capture webcam frame.
- Step 2 : Convert image into RGB format.
- Step 3 : Apply MediaPipe pose detection.
- Step 4 : Extract body landmarks.
- Step 5 : Generate body polygon.
- Step 6 : Apply gray mask.
- Step 7 : Display output.

```
Agnoctic-v32.py > _
1  import cv2
2  import mediapipe as mp
3  import numpy as np
4  mp_pose = mp.solutions.pose
5  pose = mp_pose.Pose()
6
7  cap = cv2.VideoCapture(0)
8  while True:
9      ret, frame = cap.read()
10     if not ret:
11         break
12     frame = cv2.flip(frame, 1)
13     rgb = cv2.cvtColor(
14         frame,
15         cv2.COLOR_BGR2RGB
16     )
17     results = pose.process(rgb)
18     agnoctic = frame.copy()
19     if results.pose_landmarks:
20         h, w, _ = frame.shape
21         lm = results.pose_landmarks.landmark
22         ls = lm[11]
23         rs = lm[12]
24         le = lm[13]
25         re = lm[14]
26         lw = lm[15]
27         rw = lm[16]
28         lh = lm[23]
29         rh = lm[24]
30         body = np.array([
31             [
32                 int(ls.x*w)-50,
33                 int(ls.y*h)-30
34             ],
35             [
36                 int(rs.x*w)+50,
37                 int(rs.y*h)-30
38             ],
39             [
40                 int(rw.x*w)+40,
41                 int(rw.y*h)
42             ],
43             [
44                 int(rh.x*w)+30,
45                 int(rh.y*h)+20
46             ],
47             [
48                 int(lh.x*w)-30,
49                 int(lh.y*h)+20
50             ],
51             [
52                 int(lw.x*w)-40,
53                 int(lw.y*h)
54             ]
55         ])
56         cv2.fillPoly(
57             agnoctic,
58             [body],
59             (120,120,120)
```

```

Agnostic-v32.py > ...
61     cv2.line(
62         agnostic,
63         (
64             int(ls.x*w),
65             int(ls.y*h)
66         ),
67         (
68             int(le.x*w),
69             int(le.y*h)
70         ),
71         (120,120,120),
72         50
73     )
74     cv2.line(
75         agnostic,
76         (
77             int(le.x*w),
78             int(le.y*h)
79         ),
80         (
81             int(lw.x*w),
82             int(lw.y*h)
83         ),
84         (120,120,120),
85         45
86     )
87     cv2.line(
88         agnostic,
89         (
90             int(rs.x*w),
91             int(rs.y*h)
92         ),
93         (
94             int(re.x*w),
95             int(re.y*h)
96         ),
97         (120,120,120),
98         50
99     )
100
101     cv2.line(
102         agnostic,
103         (
104             int(re.x*w),
105             int(re.y*h)
106         ),
107         (
108             int(rw.x*w),
109             int(rw.y*h)
110         ),
111         (120,120,120),
112         45
113     )
114     neck_x = int(
115         (
116             ls.x +
117             rs.x
118         )/2 * w
119     )
120
121     neck_y = int(
122         (
123             ls.y +
124             rs.y
125         )/2 * h
126     )
127     cv2.circle(
128         agnostic,
129         (
130             neck_x,
131             neck_y
132         ),
133         35,
134         (120,120,120),
135         -1
136     )
137     output = np.hstack(
138         (
139             frame,
140             agnostic
141         )
142     )
143     cv2.imshow(
144         "Original | Agnostic-v3.2",
145         output
146     )
147     key = cv2.waitKey(1)
148     if key == ord('q'):
149         break
150     cap.release()
151     cv2.destroyAllWindows()

```

Output



PARSE-AGNOSTIC-v3.2

Introduction

Parse-Agnostic-v3.2 converts the upper body region into segmentation masks. The objective is to separate clothing areas from the remaining body structure.

Unlike Agnostic-v3.2, this module produces binary mask representation instead of gray masking.

The output contains white segmented regions over black background.

Algorithm

Step 1 : Capture webcam frame.

Step 2 : Detect body landmarks.

Step 3 : Identify shoulders and hips.

Step 4 : Generate upper body mask.

Step 5 : Convert into binary image.

Step 6 : Display result.

```
59         int(rs.y*h)),
60         (int(rw.x*w),
61         int(rw.y*h)),
62         (255,255,255),
63         35
64     )
65     neckx = int(
66         ((ls.x+rs.x)/2)*w
67     )
68     necky = int(
69         ((ls.y+rs.y)/2)*h
70     )
71     cv2.circle(
72         parse,
73         (neckx,necky),
74         25,
75         (255,255,255),
76         -1
77     )
78     output = np.hstack(
79         (
80             frame,
81             parse
82         )
83     )
84     cv2.imshow(
85         "Original | Parse-Agnostic-v3.2",
86         output
87     )
88     if cv2.waitKey(1) & 0xFF == ord('q'):
89         break
90     cap.release()
91     cv2.destroyAllWindows()
```

```
1  import cv2
2  import mediapipe as mp
3  import numpy as np
4  mp_pose = mp.solutions.pose
5  pose = mp_pose.Pose()
6  cap = cv2.VideoCapture(0)
7  while True:
8      ret, frame = cap.read()
9      if not ret:
10         break
11         frame = cv2.flip(frame,1)
12         h,w,_ = frame.shape
13         rgb = cv2.cvtColor(
14             frame,
15             cv2.COLOR_BGR2RGB
16         )
17         results = pose.process(rgb)
18         parse = np.zeros(
19             (h,w,3),
20             dtype=np.uint8
21         )
22         if results.pose_landmarks:
23             lm = results.pose_landmarks.landmark
24             ls = lm[11]
25             rs = lm[12]
26             le = lm[13]
27             re = lm[14]
28             lw = lm[15]
29             rw = lm[16]
30             lh = lm[23]
31             rh = lm[24]
32             upper = np.array([
33                 [int(ls.x*w)-40,
34                  int(ls.y*h)-30],
35                 [int(rs.x*w)+40,
36                  int(rs.y*h)-30],
37                 [int(rh.x*w)+20,
38                  int(rh.y*h)],
39                 [int(lh.x*w)-20,
40                  int(lh.y*h)]
41             ])
42             cv2.fillPoly(
43                 parse,
44                 [upper],
45                 (255,255,255)
46             )
47             cv2.line(
48                 parse,
49                 (int(ls.x*w),
50                  int(ls.y*h)),
51                 (int(lw.x*w),
52                  int(lw.y*h)),
53                 (255,255,255),
54                 35
55             )
56             cv2.line(
57                 parse,
58                 (int(rs.x*w),
59                  int(rs.y*h)),
```

Output



PARSE-v3

Introduction

Parse-v3 performs body segmentation by assigning different colors to body regions. The human body is divided into semantic regions including head, torso, arms and lower body.

Different colors are used for visual representation.

Body Region Mapping

Head → Blue

Torso → Orange

Arms → Cyan

Lower Body → Green

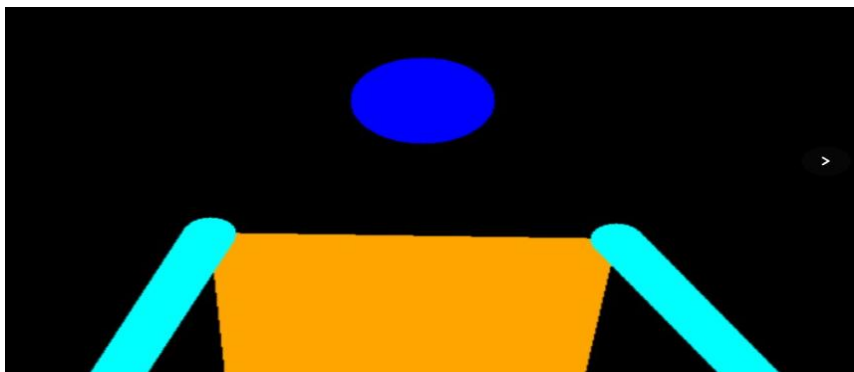
Advantages

- Easy body analysis.
- Region identification.
- Color visualization.
- Useful preprocessing stage.
- Activity Recognition
- Pose Analysis
- Virtual Try-On

Code

```
1 import cv2
2 import mediapipe as mp
3 import numpy as np
4 mp_pose = mp.solutions.pose
5 pose = mp_pose.Pose()
6 cap = cv2.VideoCapture(0)
7 while True:
8     ret, frame = cap.read()
9     if not ret:
10         break
11     frame = cv2.flip(frame,1)
12     h,w,_ = frame.shape
13     rgb = cv2.cvtColor(
14         frame,
15         cv2.COLOR_BGR2RGB
16     )
17     results = pose.process(rgb)
18     parse = np.zeros(
19         (h,w,3),
20         dtype=np.uint8
21     )
22     if results.pose_landmarks:
23         lm = results.pose_landmarks.landmark
24         nose = lm[0]
25         cv2.circle(
26             parse,
27             (
28                 int(nose.x*w),
29                 int(nose.y*h)
30             ),
31             50,
32             (255,0,0),
33             -1
34         )
35         ls = lm[11]
36         rs = lm[12]
37         le = lm[13]
38         re = lm[14]
39         lw = lm[15]
40         rw = lm[16]
41         lh = lm[23]
42         rh = lm[24]
43         lk = lm[25]
44         rk = lm[26]
45         torso = np.array([
46             [int(ls.x*w),int(ls.y*h)],
47             [int(rs.x*w),int(rs.y*h)],
48             [int(rh.x*w),int(rh.y*h)],
49             [int(lh.x*w),int(lh.y*h)]
50         ])
51         cv2.fillPoly(
52             parse,
53             [torso],
54             (0,165,255)
55         )
56         cv2.line(
57             parse,
58             (
59                 int(ls.x*w),
60                 int(rs.x*w),int(rs.y*h)],
61                 [int(rh.x*w),int(rh.y*h)],
62                 [int(lh.x*w),int(lh.y*h)]
63             )
64         )
65         cv2.line(
66             parse,
67             (
68                 int(ls.x*w),
69                 int(ls.y*h)
70             ),
71             (
72                 int(lw.x*w),
73                 int(lw.y*h)
74             ),
75             (255,255,0),
76             35
77         )
78         lower = np.array([
79             [int(lh.x*w),int(lh.y*h)],
80             [int(rh.x*w),int(rh.y*h)],
81             [int(rk.x*w),int(rk.y*h)],
82             [int(lk.x*w),int(lk.y*h)]
83         ])
84         cv2.fillPoly(
85             parse,
86             [lower],
87             (0,255,255)
88         )
89         output = np.hstack(
90             (
91                 frame,
92                 parse
93             )
94         )
95         cv2.imshow(
96             "Original | Parse-v3",
97             output
98         )
99         if cv2.waitKey(1)==ord('q'):
100             break
101         cap.release()
```

Output



DENSE POSE

Introduction

Dense Pose creates dense body representation using color mapping. Different body parts are visualized using distinct colors.

This module creates simplified dense body visualization using MediaPipe landmarks.

Working Principle

- Webcam captures image.
- Pose landmarks are extracted.
- Head region is detected.
- Torso polygon is created.
- Arms and legs are visualized.

```
densepose.py > ...
61     int(ls.x*w),
62     int(ls.y*h)
63 ),
64 (
65     int(lw.x*w),
66     int(lw.y*h)
67 ),
68 (0,255,255),
69 30
70 )
71 cv2.line(
72     dense,
73     (
74         int(rs.x*w),
75         int(rs.y*h)
76     ),
77     (
78         int(rw.x*w),
79         int(rw.y*h)
80     ),
81     (0,255,255),
82     30
83 )
84 left_leg = np.array([
85     [int(lh.x*w),int(lh.y*h)],
86     [int(lk.x*w),int(lk.y*h)],
87     [int(la.x*w),int(la.y*h)]
88 ])
89
90 right_leg = np.array([
91     [int(rh.x*w),int(rh.y*h)],
92     [int(rk.x*w),int(rk.y*h)],
93     [int(ra.x*w),int(ra.y*h)]
94 ])
95
96 cv2.polylines(
97     dense,
98     [left_leg],
99     False,
100     (255,255,0),
101     25
102 )
103
104 cv2.polylines(
105     dense,
106     [right_leg],
107     False,
108     (255,255,0),
109     25
110 )
111 out = np.hstack(
112     (
113         frame,
114         dense
115     )
116 )
117 cv2.imshow(
118     "Original | Dense Pose",
119     out
```

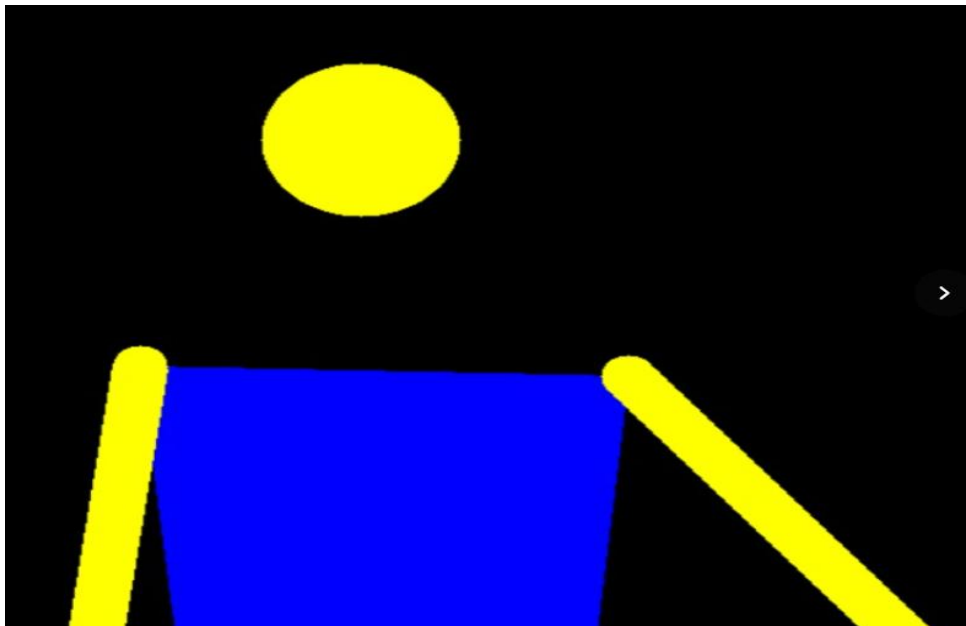
```
densepose.py > ...
1  import cv2
2  import mediapipe as mp
3  import numpy as np
4  mp_pose = mp.solutions.pose
5  pose = mp_pose.Pose()
6  cap = cv2.VideoCapture(0)
7  while True:
8      ret, frame = cap.read()
9      if not ret:
10         break
11     frame = cv2.flip(frame,1)
12     h,w,_ = frame.shape
13     rgb = cv2.cvtColor(
14         frame,
15         cv2.COLOR_BGR2RGB
16     )
17     results = pose.process(rgb)
18     dense = np.zeros(
19         (h,w,3),
20         dtype=np.uint8
21     )
22     if results.pose_landmarks:
23         lm = results.pose_landmarks.landmark
24         nose = lm[0]
25         ls = lm[11]
26         rs = lm[12]
27         le = lm[13]
28         re = lm[14]
29         lw = lm[15]
30         rw = lm[16]
31         lh = lm[23]
32         rh = lm[24]
33         lk = lm[25]
34         rk = lm[26]
35         la = lm[27]
36         ra = lm[28]
37         cv2.circle(
38             dense,
39             (
40                 int(nose.x*w),
41                 int(nose.y*h)
42             ),
43             55,
44             (0,255,255),
45             -1
46         )
47         torso = np.array([
48             [int(ls.x*w),int(ls.y*h)],
49             [int(rs.x*w),int(rs.y*h)],
50             [int(rh.x*w),int(rh.y*h)],
51             [int(lh.x*w),int(lh.y*h)]
52         ])
53         cv2.fillPoly(
54             dense,
55             [torso],
56             (255,0,0)
57         )
58         cv2.line(
59             dense,
```

```

113         frame,
114         dense
115     )
116 )
117 cv2.imshow(
118     "Original | Dense Pose",
119     out
120 )
121 if cv2.waitKey(1)==ord('q'):
122     break
123 cap.release()
124 cv2.destroyAllWindows()

```

Output



OPEN POSE

Introduction

Open Pose estimates body joints and creates skeleton representation.

Body key points are detected and connected to form pose structure.

Working Principle

- Webcam captures image.
- Body joints are detected.
- Skeleton structure is generated.
- Pose representation is displayed.

Advantages

- Real-time tracking.
- Lightweight implementation.

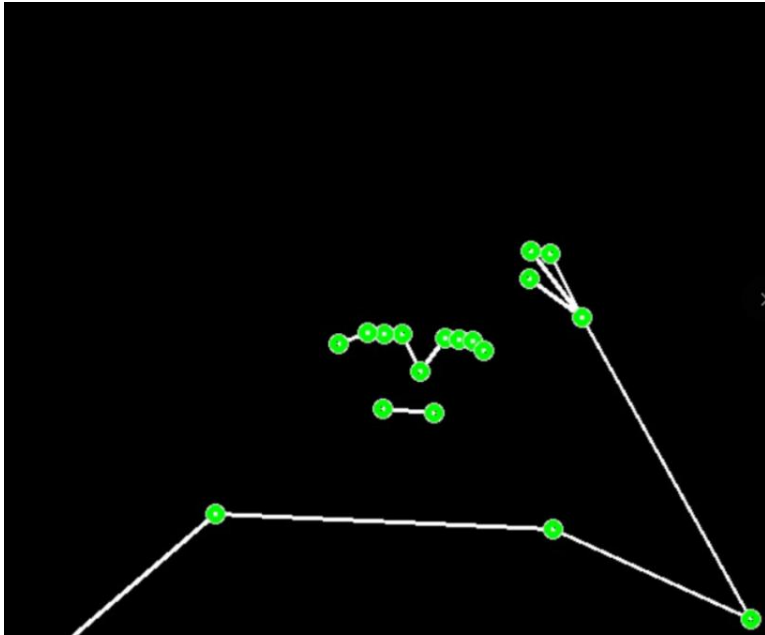
- Accurate pose estimation.
- Easy visualization.

Applications

- Gesture Recognition
- Motion Tracking
- Gaming Systems
- Human Activity Detection

Code

```
openpose.py > ...
1  import cv2
2  import mediapipe as mp
3  import numpy as np
4  mp_pose = mp.solutions.pose
5  pose = mp_pose.Pose()
6  mp_draw = mp.solutions.drawing_utils
7  cap = cv2.VideoCapture(0)
8  while True:
9      ret, frame = cap.read()
10     if not ret:
11         break
12     frame = cv2.flip(frame,1)
13     rgb = cv2.cvtColor(
14         frame,
15         cv2.COLOR_BGR2RGB
16     )
17     results = pose.process(rgb)
18     openpose = np.zeros_like(frame)
19     if results.pose_landmarks:
20         mp_draw.draw_landmarks(
21             openpose,
22             results.pose_landmarks,
23             mp_pose.POSE_CONNECTIONS,
24             mp_draw.DrawingSpec(
25                 color=(0,255,0),
26                 thickness=3,
27                 circle_radius=4
28             ),
29             mp_draw.DrawingSpec(
30                 color=(255,255,255),
31                 thickness=2
32             )
33         )
34     out = np.hstack(
35         (
36             frame,
37             openpose
38         )
39     )
40     cv2.imshow(
41         "Original | Open Pose",
42         out
43     )
44     if cv2.waitKey(1) & 0xFF == ord('q'):
45         break
46 cap.release()
47 cv2.destroyAllWindows()
```

CLOTH MASK

Introduction

Cloth Mask extracts garment region from webcam input and generates binary mask representation.

The module separates clothing from background and produces extracted cloth image.

Working Principle

- Webcam captures image.
- Pose landmarks are detected.
- Clothing region is identified.
- Cloth extraction is performed.
- Binary mask is generated.

Advantages

- Clothing extraction.
- Mask generation.
- Real-time processing.
- Useful preprocessing stage.

Code

```
cloth_mask.py > ...
1  import cv2
2  import mediapipe as mp
3  import numpy as np
4  mp_pose = mp.solutions.pose
5  pose = mp_pose.Pose()
6  cap = cv2.VideoCapture(0)
7  while True:
8      ret, frame = cap.read()
9      if not ret:
10         break
11     frame = cv2.flip(frame,1)
12     h,w,_ = frame.shape
13     rgb = cv2.cvtColor(
14         frame,
15         cv2.COLOR_BGR2RGB
16     )
17     results = pose.process(rgb)
18     mask = np.zeros(
19         (h,w),
20         dtype=np.uint8
21     )
22     cloth = np.zeros_like(frame)
23     if results.pose_landmarks:
24         lm = results.pose_landmarks.landmark
25         ls = lm[11]
26         rs = lm[12]
27         lh = lm[23]
28         rh = lm[24]
29         cloth_region = np.array([
30             [
31                 int(ls.x*w)-30,
32                 int(ls.y*h)-20
33             ],
34             [
35                 int(rs.x*w)+30,
36                 int(rs.y*h)-20
37             ],
38             [
39                 int(rh.x*w)+20,
40                 int(rh.y*h)
41             ],
42             [
43                 int(lh.x*w)-20,
44                 int(lh.y*h)
45             ]
46         ])
47         cv2.fillPoly(
48             mask,
49             [cloth_region],
50             255
51         )
52         cloth = cv2.bitwise_and(
53             frame,
54             frame,
55             mask=mask
56         )
57     mask_bgr = cv2.cvtColor(
58         mask,
59         cv2.COLOR_GRAY2BGR
60     )
61     output = np.hstack(
62         (
63             frame,
64             cloth,
65             mask_bgr
66         )
67     )
68     cv2.imshow(
69         "Original | Cloth | Cloth Mask",
70         output
71     )
72     if cv2.waitKey(1)==ord('q'):
73         break
74     cap.release()
75     cv2.destroyAllWindows()
```

Output

